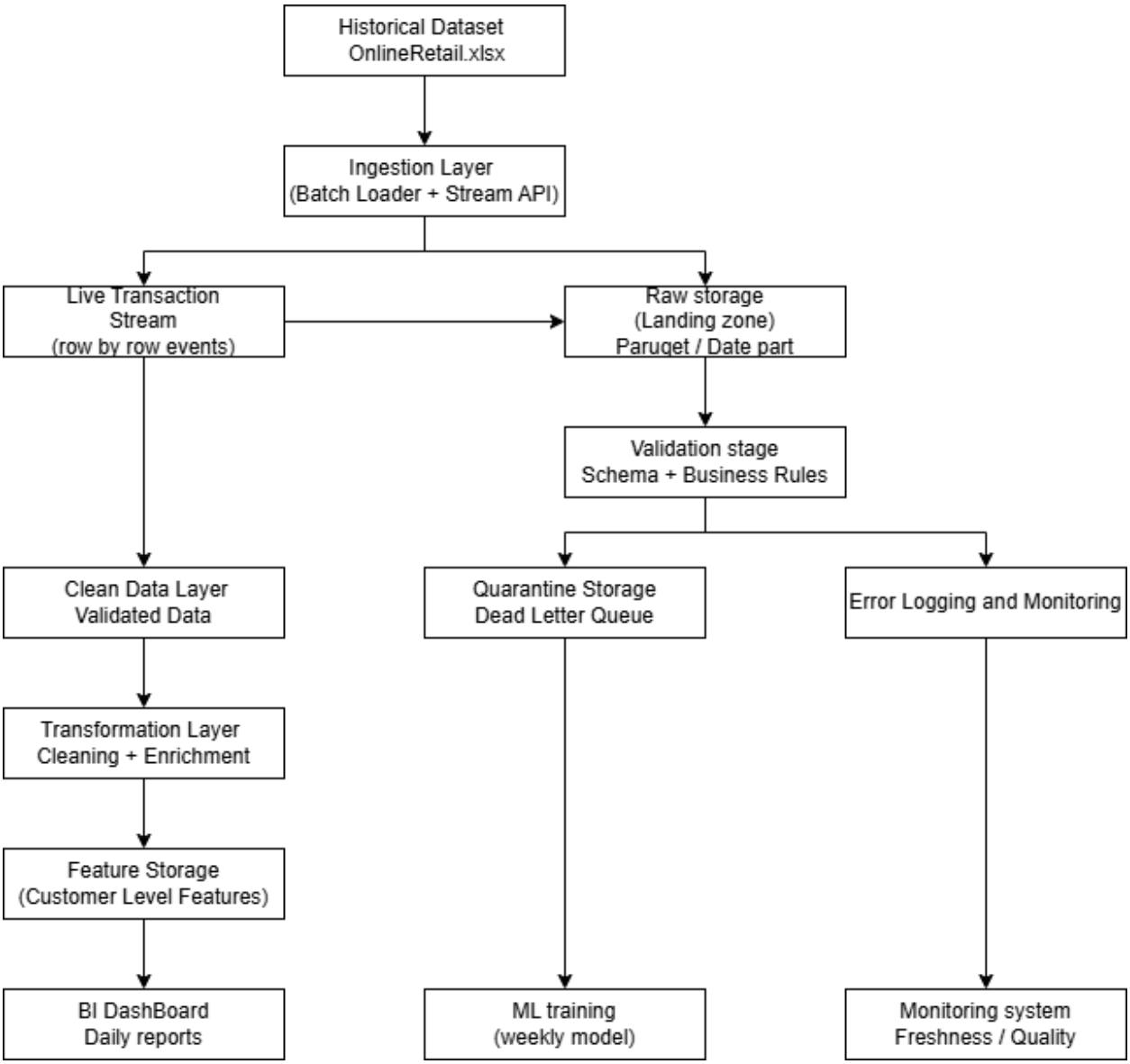
Data Pipeline Architecture Design

Task 1: Pipeline Architecture



*The schema is designed at Draw.io.

Data Sources

The pipeline receives data from two sources. The first is a historical batch dataset (`Online Retail.xlsx`) containing past transaction records. The second source is a live stream of transactions where each new order arrives as a single event. These sources simulate both historical data ingestion and real-time order processing.

Ingestion Layer

The ingestion layer is responsible for collecting data from the sources and bringing it into the pipeline. Batch ingestion loads the historical dataset once using a batch loader process. Streaming ingestion receives real-time transactions through an API or message queue. The output of this stage is raw transactional data forwarded to the landing storage.

Raw Storage (Landing Zone)

Raw storage keeps the original data exactly as it arrives from the sources. No transformations or corrections are applied at this stage. The data is stored in **Parquet format partitioned by ingestion date** for efficient querying and storage. This layer ensures reproducibility because the pipeline can always be rerun using the original data.

Validation Stage

The validation stage checks whether each record satisfies schema requirements and business rules. It verifies data types, required fields, and logical relationships between columns. Valid records proceed to the clean data layer, while invalid records are redirected to quarantine storage. This step protects downstream analytics from corrupted data.

Quarantine / Dead Letter Queue

Records that fail validation are stored in the quarantine area. This storage keeps the invalid records together with the validation error message that explains why the record failed. Data engineers can later inspect these records and decide whether they should be corrected or permanently rejected. This ensures problematic data does not contaminate the main pipeline.

Clean Data Layer

The clean layer stores validated and structured data ready for transformation. Minor formatting corrections may occur here, such as standardizing date formats or trimming text fields. Data in this layer follows the correct schema and passes validation checks. This layer acts as the trusted transactional dataset.

Transformation Layer

The transformation layer converts clean transactional data into analysis-ready datasets. It calculates derived fields such as line totals, flags cancellations, and generates date components. It also aggregates transactions into customer-level metrics like total revenue or order count. The result is structured data that supports both analytics and machine learning.

Feature Storage Layer

The feature layer stores aggregated customer-level features used for machine learning. These features include purchase frequency, spending behavior, and product diversity. Data is updated daily to reflect the newest transactions. This layer provides a consistent dataset for ML training and prediction.

Consumer Interfaces

Two main consumers use the pipeline outputs. The BI dashboard queries aggregated transactional data to produce daily reports such as sales totals and top products. The ML system reads customer-level features to train a model that predicts high-value customers. These consumers rely on the structured storage layers for reliable analytics.

Monitoring System

The monitoring system tracks pipeline health and data quality. It checks metrics such as data freshness, record volume, and schema consistency. Alerts are triggered if data stops arriving, validation failures spike, or schemas unexpectedly change. Monitoring ensures the pipeline remains reliable in production.

Task 2: Validation and Error Handling

Schema Validation Rules

1. **InvoiceNo** must be a non-empty string.
2. **StockCode** must be a valid string identifier.
3. **Description** must be a string (nullable allowed).
4. **Quantity** must be an integer.
5. **InvoiceDate** must be a valid datetime.
6. **UnitPrice** must be a numeric value.
7. **CustomerID** must be numeric or null.

8. **Country** must be a valid string.

Value Range Validation

1. **Quantity** must not equal zero.
2. **UnitPrice** must be greater than or equal to zero.
3. **InvoiceDate** cannot be in the future.
4. **CustomerID** must be positive if present.

Business Rule Validation

1. If **InvoiceNo begins with "C"**, it represents a cancellation and **Quantity must be negative**.
2. If **Quantity is negative**, the invoice must be a cancellation.
3. **LineTotal = Quantity × UnitPrice** must not be zero for normal orders.
4. A transaction should have a valid **StockCode and Description pair**.

Schema Failure

If a record fails schema validation, it is rejected immediately. The record is stored in the **quarantine table** together with an error message explaining the failure. A monitoring alert is triggered if the number of schema failures exceeds a defined threshold.

Value Range Failure

If a numeric field contains an unrealistic value, the record is flagged and moved to quarantine storage. These cases are logged for investigation because they may indicate upstream system issues. Operators can review these records and correct the source system if necessary.

Business Rule Failure

Records violating business rules are placed in the dead letter queue. These records are not processed further until the issue is resolved. After fixing the source data or adjusting rules, the quarantined records can be replayed through the validation stage.

Recovery Process

Data engineers periodically review quarantine records. If errors are corrected, the records are resubmitted to the pipeline for validation and transformation. This allows recovery without losing important data.

Task 3: Transformation and Storage

Cleaning Operations

Input: Validated transactional records
Output: Standardized dataset with corrected formats
Examples include trimming whitespace, standardizing product descriptions, and converting timestamps to a consistent timezone. These operations are idempotent because running them multiple times produces the same result.

Derived Columns

Derived fields help analytics and modeling.

Examples:

- **LineTotal = Quantity × UnitPrice**
- **IsCancellation flag**
- **InvoiceYear, InvoiceMonth, InvoiceDay**

These transformations enrich the transactional dataset and are safe to rerun.

Customer-Level Aggregations

Aggregations compute customer behavior metrics.

Examples:

- **TotalRevenue per customer**
- **OrderCount**
- **AverageOrderValue**
- **ProductDiversity (unique products purchased)**

These metrics are recalculated daily from transactional data.

ML Feature Engineering

Customer features used for prediction:

- Recency (days since last purchase)
- Frequency (number of orders)
- Monetary value (total spending)
- Product diversity

These features are calculated using a fixed observation window to avoid data leakage.

Step 3.2 Storage Layer Design

The pipeline uses three storage layers: **raw, clean, and feature**.

Raw Layer
The raw layer stores the original data exactly as it arrives from the data sources. This includes both the historical batch dataset and the live transaction stream. No transformations are applied here. The data is stored in **Parquet format** and organized by ingestion date. This layer is updated whenever new data arrives and is kept for a long time so the pipeline can be reprocessed if needed.

Clean Layer
The clean layer stores data that has already passed validation checks. At this stage, the records follow the correct schema and basic cleaning has been applied. This dataset is reliable and can be safely used for further processing. The data is also stored in **Parquet format** because it allows fast analytical queries and efficient storage. This layer is updated regularly as new validated data enters the pipeline.

Feature Layer
The feature layer stores aggregated customer-level data used for analytics and machine learning. Examples include total customer revenue, number of orders, and product diversity. These features are updated **daily** to reflect new transactions. The data is stored in **Parquet or table format** so that BI tools and ML models can easily access it.

The layered storage design is important because it separates different stages of processing. If an error occurs in the transformation step, the pipeline can restart from the clean or raw layer without losing data. This makes the system more reliable and easier to maintain.

Step 3.3 Incremental Update Strategy

The pipeline processes new data incrementally using an **ingestion timestamp as a high-water mark**. This ensures that each record is processed only once.

Late arriving records are accepted and inserted into the correct partition based on their event timestamp. Customer features are recomputed daily to incorporate new transactions. This approach keeps analytics and ML features up to date while avoiding full pipeline recomputation.